Prompt input/output reference — inputs, filled prompt, and output per call

Every LLM call in the current design, in pipeline order. For each: an **Inputs** list (field → what it is), the **filled prompt** the model receives (template variables substituted with data), and the **output** (structured-output schema + example). Generation and selection prompts read the ticket's **raw text** (`{raw text}` below = the full consolidated ticket content, ~24k tokens).

---

1. Condense (LLM ×1, per ticket)

Inputs:

- `ticketId` → the ticket id, for traceability.
- `consolidatedText` → the full raw text (description + extracted attachments, ~24k tokens).

Filled prompt (user message):

```
Ticket ID: IDMT-19761

Source material:
{raw text}
```

Output — `SummaryFields`:

```
{
  "generatedSummary": "Sales Operations needs a real-time CPQ integration to cut the enterprise
quote cycle from five days to same-day, with automated discount approval and a handoff to order
fulfilment.",
  "businessProblem": "Manual quoting delays enterprise deal closures by 3-5 days.",
  "businessCapability": "Automated real-time quote generation for enterprise accounts.",
  "keyTerms": ["CPQ", "quoting", "enterprise", "Salesforce", "Deal Desk"],
  "stakeholders": ["Sales Ops", "IT", "Finance"],
  "systemsAndProducts": ["Salesforce CPQ", "Oracle ERP", "Deal Desk Portal"]
}
```

*The summary fields are used for **retrieval** (embedding query) and routing — not fed into the generation prompts below.*

---

2. Value Stream Selection (LLM ×1, per ticket)

Inputs:

- `content` → the new ticket's **raw text** (what the model reads to decide). *(The summary is used separately as the embedding query to fetch the 6 past tickets — it is not in this prompt.)*

- `requestedCount` → how many Value Streams to return (exact; default 10).
- `historicEvidence` → the **6 similar past tickets**, each shown as its summary + the Value Streams it was tagged with (precedent).
- `candidateBlocks` → **all 50 governed Value Streams**, one compact block each.

Filled prompt (user message):

```
TICKET CONTENT:

{raw text}

REQUESTED VALUE STREAM COUNT (exact):
10

SIMILAR PAST TICKETS (evidence - the value streams these were tagged with):
- IDMT-17432: CPQ enhancements for the large Deal Desk; automated pricing configuration for
enterprise tiers.
  -> tagged value streams: Configure, Price and Quote (VSR-0042), Order Management (VSR-0017)
- IDMT-16890: Order fulfilment integration with CPQ; quote-to-order handoff latency reduction.
  -> tagged value streams: Order Management (VSR-0017)

CANDIDATE VALUE STREAMS:

Candidate: Configure, Price and Quote
entity_id: VSR-0042
description: Initiate, price and issue enterprise quotes.
category: Sales and Enrollment
trigger: A qualified enterprise opportunity needs a quote.
value: Faster, accurate quotes that shorten the sales cycle.
assumptions: Pricing is sourced from the master pricing system.

Candidate: Order Management
entity_id: VSR-0017
description: Capture and fulfil enterprise orders.
category: Fulfilment
trigger: An accepted quote becomes an order.
value: Reliable, on-time order fulfilment.
assumptions: Orders originate from approved quotes.

[... 48 more candidate blocks, all 50 governed Value Streams ...]
```

LLM output — `ValueStreamSelection`. The model emits only the four fields below, per pick:

```
{
  "picks": [
    { "entityId": "VSR-0042", "confidence": 0.91, "supportType": "direct",
      "reason": "Ticket explicitly targets CPQ automation for enterprise quoting." },
    { "entityId": "VSR-0017", "confidence": 0.74, "supportType": "implied",
      "reason": "Quote-to-order handoff implied by the fulfilment requirement." }
  ]
}
```

Resolved output — `recommendations[]`, built **deterministically after the LLM call** (no second LLM call):

```
{
  "recommendations": [
    { "valueStreamId": "VSR-0042", "valueStreamName": "Configure, Price and Quote", "confidence":
91,
      "supportType": "direct", "reason": "Ticket explicitly targets CPQ automation for enterprise
quoting.", "sourceTickets": [] },
    { "valueStreamId": "VSR-0017", "valueStreamName": "Order Management", "confidence": 74,
      "supportType": "implied", "reason": "Quote-to-order handoff implied by the fulfilment
requirement.", "sourceTickets": ["IDMT-16890"] }
  ]
}
```

Post-selection enrichment (deterministic):

- `entityId` → *resolved to* `valueStreamId` *+ the catalogue* `valueStreamName`*;* `confidence` *is scaled 0–1 → 0–100.*
- `sourceTickets` ***is back-filled, not emitted by the model***: each selected Value Stream is matched back to the **historic IDMT tickets that carried it** (the precedent from `historicEvidence`). It is populated **only for** `implied` **picks**, capped at `max_supporting_tickets` (default 2) — so an implied pick shows which past ticket(s) justify it, while a direct pick stays empty.

Human approval gate — the SME confirms the Value Stream set before any generation runs.

---

3. Stage Selection (LLM ×1, all Value Streams batched)

Inputs:

- `content` → the ticket's raw text.
- `valueStreams` → each approved Value Stream with its name, description, **valueProposition, trigger, assumptions**, and its **own candidate stages**; each stage block is `[sequence] Stage Name (stageId)`
    - description + entrance/exit criteria.

How the prompt tells the model to read stages: match the work's concrete **action** to a stage's **scope** — its description, entrance and exit criteria, value items, stakeholders — *not* on stage-name similarity; return only stage ids printed under that Value Stream; no count cap. The Value Stream's trigger and assumptions frame what that stream's work covers.

Filled prompt (user message):

```
## Ticket context
- content: {raw text}

## Approved value streams (each with its own candidate stages)
### Value stream VSR-0042
Name: Configure, Price and Quote
Description: Initiate, price and issue enterprise quotes.
Value proposition: Faster, accurate quotes that shorten the sales cycle.
Trigger: A qualified enterprise opportunity needs a quote.
Assumptions: Pricing is sourced from the master pricing system.
Candidate stages:
[1] Opportunity to Quote (VSS-0042-01)
description: Initiate and price an enterprise quote.
entrance: opportunity qualified | exit: quote issued
[2] Quote to Order (VSS-0042-02)
description: Convert an accepted quote into an order.
entrance: quote accepted | exit: order created

### Value stream VSR-0017
Name: Order Management
Description: Capture and fulfil enterprise orders.
Value proposition: Reliable, on-time order fulfilment.
Trigger: An accepted quote becomes an order.
Assumptions: Orders originate from approved quotes.
Candidate stages:
[1] Order Capture (VSS-0017-01)
description: Receive and validate the enterprise order.
entrance: order requested | exit: order validated
```

*The stage **name** is in each block (`[1] Opportunity to Quote (VSS-0042-01)`), so the model matches on scope and returns the stageId.*

Output — one entry per Value Stream:

```
{
  "valueStreams": [
    { "valueStreamId": "VSR-0042", "selectedStages": [
        { "stageId": "VSS-0042-01", "stageName": "Opportunity to Quote", "reason": "Ticket targets
the quoting initiation workflow." },
        { "stageId": "VSS-0042-02", "stageName": "Quote to Order", "reason": "Quote-to-order
handoff is in scope." } ] },
    { "valueStreamId": "VSR-0017", "selectedStages": [
        { "stageId": "VSS-0017-01", "stageName": "Order Capture", "reason": "Accepted quotes are
captured as orders." } ] }
  ]
}
```

A stage placed under the wrong Value Stream is salvaged to its owner. Empty list = "take the whole governed list for the architect to trim."

---

4. Description BODY (LLM ×1, per ticket, VS-agnostic)

Inputs:

- `content` → the ticket's raw text.

System: write the shared **structured** body; every statement must trace to a phrase in the content; availability/plan lines only when the content explicitly states them.

Filled prompt (user message):

```
Ticket context:
- content: {raw text}
```

Output — a single `{ text }` field. The structure lives **inside** the text as headed sections (`Product Availability` , the initiative + bullets, `Digital Experience` , `Integration / Operational Capabilities`), **not** as separate JSON fields. Sections with no supporting evidence are omitted — here the content states no go-live date, plans, or funding model, so the `Product Availability` block is dropped:

```
Real-Time Quote Automation:
- Integrates Salesforce CPQ with live Oracle ERP pricing, cutting the enterprise quote cycle from five days to same-day.
- Routes discounts above 20% to VP approval.

Digital Experience:
- Faster self-serve quoting for enterprise sales reps.

Integration / Operational Capabilities:
- Salesforce CPQ to Oracle ERP pricing integration.
- Accepted quotes hand off to order fulfilment within the Deal Desk's 4-hour SLA.
```

---

5. Description FRAMING (LLM ×1, all Value Streams batched)

Inputs:

- `content` → the ticket's raw text.
- `valueStreams` → each approved Value Stream: `valueStreamId, valueStreamName, valueStreamDescription, valueProposition` .

Filled prompt (user message):

```
Ticket context:
- content: {raw text}

Approved value streams:
- valueStreamId: VSR-0042
  valueStreamName: Configure, Price and Quote
  valueStreamDescription: Initiate, price and issue enterprise quotes.
  valueProposition: Faster, accurate quotes that shorten the sales cycle.
- valueStreamId: VSR-0017
  valueStreamName: Order Management
  valueStreamDescription: Capture and fulfil enterprise orders.
  valueProposition: Reliable, on-time order fulfilment.
```

Output — `framings[]`:

```
{
  "framings": [
    { "valueStreamId": "VSR-0042", "text": "Within Configure, Price and Quote, this initiative
makes enterprise quoting real-time and rule-driven." },
    { "valueStreamId": "VSR-0017", "text": "For Order Management, it ensures accepted quotes flow
cleanly into fulfilment." }
  ]
}
```

Final per-VS description = its framing paragraph + the shared body, concatenated deterministically.

---

6. Business Needs (LLM ×1 per Value Stream)

Inputs:

- `valueStream` → `valueStreamId, valueStreamName, valueStreamDescription, valueProposition`.
- `selectedStages` → that VS's selected stages (`[seq] Stage Name (stageId)` + description + criteria).
- `content` → the ticket's raw text.

System: write the consolidated Business Needs; every need, dependency, and rule must trace to a phrase in the content; one "Value Stage:" block per stage.

Filled prompt (user message):

```
## Approved value stream
ID: VSR-0042
Name: Configure, Price and Quote
Description: Initiate, price and issue enterprise quotes.
Value proposition: Faster, accurate quotes that shorten the sales cycle.

## Selected stages (write needs for these stages only)
[1] Opportunity to Quote (VSS-0042-01)
description: Initiate and price an enterprise quote. entrance: opportunity qualified | exit: quote
issued
[2] Quote to Order (VSS-0042-02)
description: Convert an accepted quote into an order. entrance: quote accepted | exit: order
created

## Ticket context
- content: {raw text}
```

Output — `{ text }` (one consolidated document):

```
Value Stage: Opportunity to Quote

Business Product Feature: Real-Time Quoting
1. Sales Operations requires real-time quote generation to replace the current 3-5 day manual
cycle.
   Dependency: Salesforce CPQ and the Oracle pricing API.
   Business Rule: Discounts above 20% require VP approval.

Value Stage: Quote to Order

Business Product Feature: Order Handoff
1. Accepted quotes must hand off to order fulfilment within the Deal Desk 4-hour SLA.
```

---

7. Capabilities (L3) (LLM ×1, ALL Value Streams merged)

Inputs:

- `content` → the ticket's raw text.
- `valueStreams` → grouped **Value Stream → Stage → governed candidate L3**. Each Value Stream carries its name, description, **valueProposition, trigger, assumptions**; each candidate L3 line is `capabilityId | name - description [tier] (L2: parent)`.

System: for each stage, select L3 only from THAT stage's printed candidate list; strict stage isolation (the same id/name repeats across stages and Value Streams — match the exact printed id only). **L2 is derived 1-1 from the selected L3 — no separate L2 call.**

Filled prompt (user message):

```
## Ticket context
- content: {raw text}

## Approved value streams, each with its selected stages and each stage's own candidate L3
### Value Stream VSR-0042 — Configure, Price and Quote
description: Initiate, price and issue enterprise quotes.
value proposition: Faster, accurate quotes that shorten the sales cycle.
trigger: A qualified enterprise opportunity needs a quote.
assumptions: Pricing is sourced from the master pricing system.

### Stage VSS-0042-01
[1] Opportunity to Quote (VSS-0042-01)
description: Initiate and price an enterprise quote. entrance: opportunity qualified | exit: quote
issued
Candidate L3 capabilities (choose by id; each shows its parent L2):
- L3-0042-0011 | Real-Time Pricing Engine Integration - live CPQ pricing feed [tier: core] (L2:
Pricing and Discounting)
- L3-0042-0012 | Automated Discount Approval Workflow - routes discounts for approval (L2:
Approval and Governance)

### Stage VSS-0042-02
[2] Quote to Order (VSS-0042-02)
description: Convert an accepted quote into an order. entrance: quote accepted | exit: order
created
Candidate L3 capabilities (choose by id; each shows its parent L2):
- L3-0042-0021 | Order Handoff Orchestration - passes the quote to fulfilment (L2: Order
Management)

### Value Stream VSR-0017 — Order Management
description: Capture and fulfil enterprise orders.
value proposition: Reliable, on-time order fulfilment.
trigger: An accepted quote becomes an order.
assumptions: Orders originate from approved quotes.

### Stage VSS-0017-01
[1] Order Capture (VSS-0017-01)
description: Receive and validate the enterprise order. entrance: order requested | exit: order
validated
Candidate L3 capabilities (choose by id; each shows its parent L2):
- L3-0017-0011 | Order Intake Automation - captures order data (L2: Order Management)
```

Output — keyed by stageId (re-attributed to each Value Stream afterward):

```
{
  "stages": [
    { "stageId": "VSS-0042-01", "capabilities": [
        { "capabilityId": "L3-0042-0011", "name": "Real-Time Pricing Engine Integration",
"reason": "Live CPQ pricing feed from Oracle ERP." },
        { "capabilityId": "L3-0042-0012", "name": "Automated Discount Approval Workflow",
"reason": "Discounts above 20% require VP approval." } ] },
    { "stageId": "VSS-0042-02", "capabilities": [
        { "capabilityId": "L3-0042-0021", "name": "Order Handoff Orchestration", "reason":
"Accepted quotes hand off to fulfilment." } ] },
    { "stageId": "VSS-0017-01", "capabilities": [
        { "capabilityId": "L3-0017-0011", "name": "Order Intake Automation", "reason": "Captures
the order on quote acceptance." } ] }
  ]
}
```


*Strict isolation + salvage keep each L3 under its owning stage. **L2** = the unique parent L2s of the selected L3, derived per Value Stream.*

Final Theme package (deterministic — no LLM)

One package per approved Value Stream:


```
{
  "themeTitle": "Enabling Real-Time Quote Automation -- Configure, Price and Quote",
  "themeDescription": "<framing + body>",
  "selectedStages": [ { "stageId": "VSS-0042-01", "stageName": "Opportunity to Quote", "reason":
"..." } ],
  "businessNeeds": "<consolidated text>",
  "l3Capabilities": [ { "stageId": "VSS-0042-01", "capabilities": [ ... ] } ],
  "l2Capabilities": [ { "stageId": "VSS-0042-01", "capabilities": [ { "capabilityId": "L2-0042-
001", "name": "Pricing and Discounting" } ] } ]
}
```

Schema conventions (locked)

- **Casing:** all output fields serialize **camelCase** on the wire (`valueStreamId` , `stageName` , `capabilityId`). Pydantic models are snake_case internally.
- **Names are canonical from the catalogue.** The model *echoes* `stageName` / capability `name` as a selection anchor, but on resolve we **overwrite them with the governed catalogue name** — consumers trust the resolved value, not the model's echo. (Capability uses `name` ; stage uses `stageName` .)
- `confidence` **is 0–100** (the model emits 0–1; selection scales it).
- **Empty** `reason` **is valid**, not an error: a salvaged capability/stage (reassigned to its true owner) and every derived **L2** carry `reason: ""` because they weren't reasoned in place.
- `supportType` is the enum `direct | implied` ; `sourceTickets` is populated only for `implied` .

Inputs sent to each call (Value Stream catalogue fields)

The governed Value Stream catalogue has: `name, id, description, valueProposition, trigger, category, assumptions, definedTerms, stakeholders`. What each call currently passes:

call	VS fields sent	not sent
VS Selection (candidate block)	name, id, description, category, trigger, valueProposition, assumptions	definedTerms, stakeholders
Stage Selection	name, id, description, valueProposition, **trigger, assumptions**	category, definedTerms, stakeholders
Capabilities (merged)	name, id, description, **valueProposition, trigger, assumptions**	category, definedTerms, stakeholders
Business Needs	name, id, description, valueProposition	assumptions, trigger, category

Stage candidates already carry: `stageName, description, entrance/exit criteria, valueItems, stakeholders`.

Recent enrichment: `trigger` + `assumptions` were added to **Stage Selection** and **Capabilities** (and `valueProposition` to Capabilities), so the model has the Value Stream's intended scope when mapping work → stage → L3. *(Re-run* `eval_stages` / `eval_l3` *to confirm the lift before final lock.)* Business Needs still omits assumptions/trigger — add there too if the enrichment proves out.